

Chapter 1: Introduction to Python

Python is a high-level, general-purpose programming language known for its readable syntax. It is widely used in web development, data science, automation, and artificial intelligence. This guide walks through the core concepts step by step, mixing explanations with small code examples.

Practice this concept with your own examples before moving to the next chapter.

Chapter 2: Variables and Data Types

A variable is a name that stores a value. Python figures out the type automatically, so you do not need to declare it. Common types are integers, floats, strings, and booleans.

```
name = "Vikram"  
age = 24  
height = 5.9  
is_student = True  
  
print(type(name))    # <class 'str'>  
print(type(age))     # <class 'int'>
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 3: Strings

Strings hold text. You can join them, slice them, and change their case using built-in methods. F-strings are the easiest way to insert variables into text.

```
greeting = "hello world"
print(greeting.upper())
print(greeting[0:5])

name = "Python"
print(f"I am learning {name}")
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 4: Lists

A list is an ordered collection that can be changed. Lists can hold any type of value, and you can add, remove, or sort items easily.

```
fruits = ["apple", "banana", "mango"]
fruits.append("grape")
fruits.remove("banana")
fruits.sort()
print(fruits)
print(len(fruits))
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 5: Tuples and Sets

Tuples are like lists but cannot be changed after creation, which makes them safe for fixed data. Sets store only unique values and are useful for removing duplicates or checking membership quickly. Choosing the right collection makes code clearer and faster.

Practice this concept with your own examples before moving to the next chapter.

Chapter 6: Dictionaries

A dictionary stores data as key-value pairs. It is perfect for looking things up by name instead of by position.

```
student = {"name": "Asha", "grade": "A", "age": 21}
print(student["name"])
student["age"] = 22

for key, value in student.items():
    print(key, "->", value)
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 7: Conditionals

Conditionals let a program make decisions. Python uses if, elif, and else, and indentation defines which lines belong to each block.

```
marks = 78

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 8: Loops

Loops repeat actions. A for loop goes through each item in a collection, while a while loop runs as long as a condition is true.

```
for i in range(5):  
    print("Count:", i)
```

```
n = 3  
while n > 0:  
    print(n)  
    n -= 1
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 9: Why Clean Code Matters

Clean code is easier to read, test, and maintain. Use meaningful variable names, keep functions short, and avoid repeating yourself. Comments should explain why something is done, not what the code already says. Teams that write clean code spend less time fixing bugs and more time building features. Following the PEP 8 style guide is a good habit for every Python developer.

Practice this concept with your own examples before moving to the next chapter.

Chapter 10: Functions

Functions group reusable code under a name. They can take inputs, called parameters, and send back a result using return.

```
def add(a, b):  
    return a + b  
  
def greet(name="friend"):  
    return f"Hello, {name}!"  
  
print(add(3, 4))  
print(greet("Riya"))
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 11: Lambda and List Comprehension

Lambda functions are small one-line functions. List comprehensions build new lists in a single readable line.

```
square = lambda x: x * x
print(square(6))

nums = [1, 2, 3, 4, 5]
evens = [n for n in nums if n % 2 == 0]
print(evens)
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 12: Classes and Objects

Object-oriented programming groups data and behaviour together. A class is a blueprint, and an object is one instance built from it.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says woof"

d = Dog("Bruno")
print(d.bark())
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 13: Inheritance

Inheritance lets one class reuse and extend another class. The child class gets all methods of the parent and can override them.

```
class Animal:
    def speak(self):
        return "Some sound"

class Cat(Animal):
    def speak(self):
        return "Meow"

print(Cat().speak())
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 14: Error Handling

Errors happen when something unexpected occurs, like dividing by zero or opening a missing file. The try and except blocks let the program handle errors gracefully instead of crashing.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 15: Working with Files

Python can read and write files easily. Using the with statement closes the file automatically when you are finished.

```
with open("notes.txt", "w") as f:  
    f.write("First line\n")  
  
with open("notes.txt", "r") as f:  
    print(f.read())
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 16: Modules and Packages

A module is simply a Python file with functions and variables that other files can import. Packages are folders of modules. The Python Standard Library already includes many useful modules such as `math`, `datetime`, `os`, and `json`. External packages are installed with `pip`, the Python package manager, and they make it possible to build powerful applications quickly.

Practice this concept with your own examples before moving to the next chapter.

Chapter 17: Using the math and random Modules

Built-in modules save time. The math module gives mathematical functions, and random helps generate random values.

```
import math
import random

print(math.sqrt(49))
print(math.pi)
print(random.randint(1, 10))
print(random.choice(["red", "blue", "green"]))
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 18: Working with JSON

JSON is a common format for sharing data between programs and web services. Python converts between JSON text and dictionaries using the `json` module.

```
import json

data = {"city": "Pune", "temp": 31}
text = json.dumps(data)
print(text)

back = json.loads(text)
print(back["city"])
```

Practice this concept with your own examples before moving to the next chapter.

Chapter 19: Introduction to AI with Python

Python is the most popular language for artificial intelligence. Libraries like NumPy, pandas, scikit-learn, and LangChain make it simple to process data, train models, and build applications on top of large language models. Retrieval Augmented Generation, or RAG, is a technique where documents are split into chunks, stored in a vector database, and retrieved to give the model relevant context before it answers a question.

Practice this concept with your own examples before moving to the next chapter.

Chapter 20: Summary

In this guide you learned variables, strings, collections, conditionals, loops, functions, classes, error handling, files, modules, and JSON. Practice by building small projects, and keep reading other people's code.

```
def summary():
    topics = ["variables", "loops", "functions", "classes"]
    for t in topics:
        print(f"Learned: {t}")

summary()
```

Practice this concept with your own examples before moving to the next chapter.